Scordatura Assistant

Download latest release — grab `ScordaturaAssistant.zip` from the Assets section.

Takes a MusicXML part written in **sounding pitches** and produces a 6-staff comparison score:

1. **Sounding (source)** — the original, untouched.
2. **Favor upper** — string assignments rendered as the player would read them; where a note can be played on more than one string, it is assigned to the highest-pitched string available (e.g. E string on violin, A string on cello — strings notated I and II).
3. **Favor lower** — same, but assigning to the lowest-pitched string available (e.g. G/C string — strings notated III and IV).
4. **Hybrid (upper+lower)** — event by event, whichever of staves 2/3's own choice is locally better once the upper/lower bias is set aside, so it's a literal note-by-note combination of the two.
5. **No preference** — solved independently with no string-index bias at all; open strings, string hints, and smooth position changes are the only things steering it.
6. **Favor middle** — solved independently, preferring strings near the center of the fingerboard.

Each note in staves 2-6 gets a string indication (Roman numeral in most engravers) and is re-pitched to what the performer fingers given the scordatura. Staves 4-6 are exploratory extra options — see How the solver chooses — and may be trimmed down later if some of them don't earn their keep in practice.

Install

Requires Python 3.9+. If Python is not already on your machine:

- **macOS**: python.org/downloads or `brew install python`
- **Windows**: python.org/downloads — check "Add Python to PATH" during setup.

Run (local web app — macOS)

1. Place the `ScordaturaAssistant` folder anywhere.
2. Double-click `launch.command`. The first time macOS may warn you it's from an unidentified developer — right-click it and choose **Open**, then confirm. On first launch it will create a virtual environment and install dependencies automatically (requires an internet connection); subsequent launches start immediately.
    - If you see a permissions error on first launch, go to **System Settings → Privacy & Security → Files and Folders** and grant Terminal access to the folder where the app is stored. Alternatively, grant Terminal **Full Disk Access** under Privacy & Security.

A browser tab opens at `http://127.0.0.1:5005`. Drag-drop a MusicXML file, pick an instrument preset, edit base pitches / offsets / max-fret if needed, tick **Translate 24EDO** if your score uses quarter-

tone accidentals (see Microtonal accidentals), hit Translate, and the 6-staff output downloads. Close the Terminal window to stop the server.

To update to a newer version, delete `~/.scordatura` in Terminal and relaunch:

```
rm -rf ~/.scordatura
```

Run (local web app — Windows)

1. Place the `ScordaturaAssistant` folder anywhere (e.g. `Documents`).
2. Double-click `launch.bat`. Windows SmartScreen may warn you — click **More info → Run anyway**. On first launch it will create a virtual environment and install dependencies automatically (requires an internet connection); subsequent launches start immediately.

A browser tab opens at `http://127.0.0.1:5005`. Close the Command Prompt window to stop the server.

To update to a newer version, download the latest `ScordaturaAssistant.zip` from the releases page and replace your old `ScordaturaAssistant` folder with the new one (the Windows launcher runs the code in its own folder, so this is what installs the new version). If the app fails to start after updating, delete the old environment and relaunch `launch.bat`. In Command Prompt, from inside the `ScordaturaAssistant` folder:

```
rmdir /s /q .venv
```

Run (CLI / advanced)

```
python3 -m venv .venv
.venv/bin/pip install -r requirements.txt   # macOS / Linux
.venv\Scripts\pip install -r requirements.txt  # Windows

.venv/bin/python -m scordatura \        # macOS / Linux
    --input  path/to/score.musicxml \
    --output path/to/out.musicxml \
    --config path/to/tuning.json

.venv\Scripts\python -m scordatura \   # Windows
    --input  path/to/score.musicxml \
    --output path/to/out.musicxml \
    --config path/to/tuning.json
```

Add `--translate-24edo` to keep quarter-tone accidentals (see Microtonal accidentals); without it every microtonal accidental is rounded to a semitone.

Open the output in MuseScore, Dorico, or Finale.

Run (local web app — command line)

```
.venv/bin/python -m scordatura.web    # macOS / Linux
.venv\Scripts\python -m scordatura.web  # Windows
```

Auto-opens a browser at `http://127.0.0.1:5005`. Ctrl-C in the terminal to stop the server.

Config (JSON)

```
{
  "instrument": "violin",
  "fingerboard": {
    "n_strings": 4,
    "base_pitches": [55, 62, 69, 76],
    "offsets":      [ 2,  2,  0, -4],
    "max_fret":     [19, 19, 19, 24]
  },
  "solver": {
    "alpha": 0.05,
    "beta":  0.05,
    "beam":  null
  }
}
```

Field	Meaning
`instrument`	`violin`, `viola`, `cello`, `contrabass`, `hardanger`, `fiddle`
`n_strings`	4 or 5
`base_pitches`	Normal open-string MIDI, low → high (e.g. violin = `[55, 62, 69, 76]`) — see String numbering for how this maps to notated string numbers
`offsets`	Per-string semitone scordatura. `-4` retunes that string down a major third; `0` leaves it standard
`max_fret`	Per-string semitone reach above the open string. `24` = two octaves
`alpha`	Position-shift weight in transitions (small)
`beta`	String-change weight in transitions (small)

Field	Meaning
`beam`	Optional beam-search width; `null` = exact Viterbi

MIDI cheat sheet: C2=36, G2=43, D3=50, A3=57, E4=64, A4=69, E5=76. So a cello tuned C–G–D–A is `[36, 43, 50, 57]`.

Scordatura math

The player reads `sounding − offset`. A C-string tuned down a semitone (offset `−1`) reading a sounding C produces a notated C♯, because the string sounds a semitone lower than the player's fingers expect.

String numbering

`<string>` markings — both on input (explicit/persisted string hints, see below) and on output (the Roman-numeral string indications written onto staves 2 & 3) — follow the **standard MusicXML/notation convention**: string **1** is always the *highest-pitched* string, counting downward to the lowest. On violin that's E = 1, A = 2, D = 3, G = 4; on cello/viola it's the same idea, just fewer or differently-tuned strings.

This is the mirror image of `base_pitches` / `offsets` / `max_fret` in the JSON config, which are always ordered **low → high** by array index (index 0 = lowest string, since `base_pitches` must be sorted ascending). So for a standard violin config `base_pitches: [55, 62, 69, 76]` (G, D, A, E), array index 3 (E, the last entry) is notated string **1**, and array index 0 (G, the first entry) is notated string **4**.

How the solver chooses

Phrase-level Viterbi over the whole part. For each event it minimizes:

- **Emission**: strategy preference (upper vs. lower string) plus an open-string bonus, **both weighted by note duration**. A long sustained note that lands on its open string scores heavily; a 32nd-note barely notices.
- **Transition**: small penalties for jumping fret positions or changing strings between consecutive events.
- **Hard constraints**: a pitch can't be assigned to a string out of its range, and two pitches with overlapping sustain can't share a string.

Open strings are a *soft* preference, not a hard pin — when a later note needs the open string for a tie or unison, the solver can sacrifice an earlier open assignment.

"Favor upper" is solved first, then "favor lower" is solved with a penalty (`duplicate_penalty`) against occupying the exact same *set* of strings "favor upper" used for that event, so long as a genuinely different set of strings is reachable for it. This is judged per event, not per pitch — a chord/double-stop can't dodge the penalty by simply swapping which pitch uses which string while still occupying the same pair of strings overall. This keeps the two staves from silently converging on the identical fingering — without

ever overriding an explicit/persisted string marking (see below), and without forcing a duplicate-avoiding choice when no alternative set of strings is actually reachable.

Combined/exploratory alternatives

Three further staves offer alternatives that sit between "favor upper" and "favor lower", for cases where the best fingering is neither extreme. None of them apply `duplicate_penalty` — they're free to converge with any other staff wherever that's genuinely the best choice:

- **Hybrid (upper+lower)** doesn't explore the fingerboard afresh. Instead, at each event it's restricted to exactly the two candidates "favor upper" and "favor lower" already chose there, and picks between them using the normal emission cost with the upper/lower bias switched off (so only open-string bonus, string hints, and duration weighting decide) — the usual transition cost (`alpha`/`beta`) still discourages switching sides needlessly often, so it doesn't flip-flop note to note without reason. Where the two staves already agree on a pitch, there's nothing to choose between and it just inherits that choice.
- **No preference** is a completely independent solve (full fingerboard, not restricted to staves 2/3's choices) with the upper/lower bias set to zero throughout — string choice is driven purely by open strings, hints, and smooth transitions.
- **Favor middle** is also a fully independent solve, but with an explicit bias toward strings near the center of the fingerboard, rather than either edge.

Artificial harmonics

To mark a chord as an artificial harmonic in the source:

- Encode it as a 2-note chord (fundamental + touched note a perfect 4th above), AND
- Either attach a `Harmonic` articulation to the chord, OR set the touched pitch's notehead to `diamond`.

The fundamental's pitch drives string selection. Output gets a `StringHarmonic` (artificial) articulation and a diamond notehead on the touched note.

Plain P4 double-stops without the explicit marker stay as regular chords.

String indications persist within a voice

If a source note carries an explicit string number (a `<technical><string>` marking, or a Dorico-style `<other-technical smufl="guitarStringN">` marker — both are recognized, and both use the numbering convention above: 1 = highest string), the solver treats that string as strongly preferred not just for that note, but for every later note in the same voice, until a new explicit marking appears. This lets you mark a string change once at the start of a passage instead of on every note.

The preference is soft: if the marked string is genuinely unplayable at some point (out of range, or already occupied by a sustained note), the solver reports a warning and falls back to another string for that note,

without breaking the persisted marking for the notes that follow.

Markings persist per voice, so a separate voice (e.g. a down-stem line sharing the staff) can carry its own independent string marking without affecting the other voice's persisted string.

See `examples/StringVoicingChallenge.musicxml` for a worked example, including the intended "as fingered" result.

Microtonal accidentals and HEJI

The **Translate 24EDO** checkbox in the web app (CLI: `--translate-24edo`) decides what happens to microtonal accidentals. It is **off by default**.

Setting	What the output contains
Off (default)	Every microtonal accidental is rounded to the nearest 12-EDO semitone. Exact quarter-tones round *up* (quarter-sharp becomes sharp, quarter-flat becomes natural, three-quarters-sharp becomes double-sharp, three-quarters-flat becomes flat), and quarter-tone accidental names are rewritten to match, so no quarter-tone accidental appears anywhere in the output.
On	Standard quarter-tone accidentals (half-sharp, half-flat, three-quarters) are kept and respelled when a string's scordatura offset transposes them. Any other fractional alter is snapped first: to a quarter-tone if it lies within 0.1 of one, otherwise to the nearest semitone.

Use it off for HEJI and other just-intonation scores. Their commas are written by Dorico as small fractional alters (for example `0.903` or `-0.266`). Left on, these would be read as quarter-tones, making the music look like 24-EDO. Turn it **on** only for music that genuinely uses quarter-tone accidentals.

HEJI is never preserved, in either setting. Dorico exports HEJI accidentals as bare fractional `<alter>` values with no accidental glyph, draws no accidental for such values when it imports MusicXML, and ignores SMuFL glyph names, so there is no way to carry the glyphs through MusicXML. Repaint HEJI in your engraver after importing the translated score. The tool prints how many microtonal values it snapped.

Limitations

- **Microtonal accidentals.** HEJI commas and other just-intonation accidentals are not preserved; they are rounded to semitones (or, with Translate 24EDO on, to quarter-tones). See Microtonal accidentals and HEJI.
- **Voice round-trips** in MusicXML are fragile: keep each measure either fully voiced or fully non-voiced. Mixing the two in one measure can confuse engravers downstream. A measure with only one voice is treated as voice "1" for the purposes of string-indication persistence and tie continuity, even if the source used a different explicit voice number for it.

Tuning the solver

If the output puts long notes on the wrong string:

- Raise `open_string_bonus` magnitude (currently `-3.0` per quarter-note in `solver.py`) to bias more strongly toward open strings.
- Raise `beta` to discourage string-changes within a passage.
- Tighten `max_fret` per string to forbid impractical positions.
- Adjust `string_hint_bonus` magnitude (currently `-1000.0`, flat rather than duration-weighted, in `solver.py`) to change how strongly an explicit/persisted string marking outweighs other preferences.
- Adjust `duplicate_penalty` magnitude (currently `500.0`, flat rather than duration-weighted, in `solver.py`) to change how strongly "favor lower" avoids repeating "favor upper"'s overall per-event string usage.
- The three combined/exploratory staves reuse `alpha`/`beta`/`open_string_bonus`/etc., so tuning those also reshapes "Hybrid", "No preference", and "Favor middle" — there's no separate knob for them yet.

Files

- `scordatura/` — the package (`fingerboard`, `events`, `solver`, `spelling`, `render`, `config`, `cli`).
- `fixtures/` — example MusicXML inputs and configs.
- `scripts/build_fixture.py` — regenerates the synthetic test score.
- `scripts/smoke_test.py` — end-to-end sanity check.